

torch.fft.ihfft2#

<https://pytorch.org/docs/stable/generated/torch.fft.ihfft2.html>

Description:

Computes the 2-dimensional inverse discrete Fourier transform of real input. Equivalent to `ihfftn()` but transforms only the two last dimensions by default. Supports `torch.half` on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions. `input` (Tensor) – the input tensor `s` (Tuple[int], optional) – Signal size in the transformed dimensions. If given, each dimension `dim[i]` will either be zero-padded or trimmed to the length `s[i]` before computing the Hermitian IFFT. If a length `-1` is specified, no padding is done in that dimension. Default: `s = [input.size(d) for d in dim]`

Function Signature

```
torch.fft.ihfft2(input, s=None, dim=(-2, -1), norm=None, *,  
out=None) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

out (Tensor, optional) – the output tensor.

Code Examples

```
>>> T = torch.rand(10, 10)
>>> t = torch.fft.ihfft2(t)
>>> t.size()
torch.Size([10, 6])
```

```
>>> fftn = torch.fft.ifft2(t)
>>> torch.allclose(fftn[..., :6], rfftn)
True
```

```
>>> two_ffts = torch.fft.ifft(torch.fft.ihfft(t, dim=1), dim=0)
>>> torch.allclose(t, two_ffts)
True
```

Notes & Warnings

NOTE

Note Supports torch.half on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions.

Detailed Documentation

Documentation

Computes the 2-dimensional inverse discrete Fourier transform of real input. Equivalent to `ihfft()` but transforms only the two last dimensions by default.

Supports torch.half on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions.

`input (Tensor)` – the input tensor

`s (Tuple[int], optional)` – Signal size in the transformed dimensions. If given, each dimension `dim[i]` will either be zero-padded or trimmed to the length `s[i]` before computing the Hermitian IFFT. If a length -1 is specified, no padding is done in that dimension. Default: `s = [input.size(d) for d in dim]`

`dim (Tuple[int], optional)` – Dimensions to be transformed. Default: last two dimensions.

`norm (str, optional)` –

Normalization mode. For the backward transform (`ihfft2()`), these correspond to:

"forward" - no normalization

"backward" - normalize by $1/n$

"ortho" - normalize by $1/\sqrt{n}$ (making the Hermitian IFFT orthonormal)

Where $n = \text{prod}(s)$ is the logical IFFT size. Calling the forward transform (`hfft2()`) with the same normalization mode will apply an overall normalization of $1/n$ between the two transforms. This is required to make `ihfft2()` the exact inverse.

Default is "backward" (normalize by $1/n$).

`out` (Tensor, optional) – the output tensor.

Compared against the full output from `ifft2()`, the Hermitian time-space signal takes up only half the space.

The discrete Fourier transform is separable, so `ihfft2()` here is equivalent to a combination of `ifft()` and `ihfft()`: